SYNTH-03: HTTP Monitors

> **Series:** SYNTH — Synthetic Monitoring | **Notebook:** 3 of 6 | **Created:** December 2025 | **Last Updated:** 10/02/2026

Lightweight API and Endpoint Monitoring

This notebook covers HTTP monitors for API health checks, endpoint validation, and multi-step API workflows using the latest Dynatrace platform.

Table of Contents

1. HTTP Monitor Overview
2. Single Request Monitors
3. Multi-Step HTTP Monitors
4. Authentication
5. Response Validation
6. SSL Certificate Monitoring
7. Analyzing HTTP Results

Prerequisites

- ✅ Access to a Dynatrace environment with Synthetic Monitoring
- ✅ Completed SYNTH-01 Fundamentals
- ✅ API endpoint(s) to monitor

> **Data model:** HTTP results live in `fetch dt.synthetic.events`. Monitor-level rows (`event.type == "http_monitor_execution"`) carry `result.state` and total `result.statistics.duration`; per-request rows (`event.type == "http_step_execution"`) carry the timing breakdown (DNS / TCP / TLS / TTFB) and SSL certificate data.

1. HTTP Monitor Overview

HTTP monitors execute lightweight HTTP requests without browser overhead:

HTTP vs Browser Monitors

Aspect	HTTP Monitor	Browser Monitor
Execution	Direct HTTP call	Full browser
Speed	Fast (< 1s typical)	Slower (3-30s)
Resources	Minimal	Chrome instance
JavaScript	Not executed	Fully executed
Frequency	1 min and up (to 4 h), or on demand	5, 10, 15, 30 min; 1, 2, 4 h; or on demand
Cost	Lower	Higher

Use Cases

Scenario	HTTP Monitor Type
API health check	Single request
REST API endpoint	Single request
Auth token + API call	Multi-step
GraphQL queries	Single/Multi-step
Webhook testing	Single request

Scenario	HTTP Monitor Type
Certificate expiry	Single request + SSL check

Configuration Path

Dynatrace menu → Synthetic → Create synthetic monitor → Create HTTP monitor

2. Single Request Monitors

Request Configuration

Setting	Description	Example
URL	Full endpoint URL	`https://api.example.com/health`
Method	HTTP verb	GET, POST, PUT, DELETE, PATCH
Headers	Custom headers	`Authorization: Bearer ...`
Body	Request payload	JSON, form data, raw
Performance threshold	Per-request threshold that can raise a problem	Set in the request's settings

Common HTTP Methods

Method	Use Case	Body
`GET`	Retrieve data	None
`POST`	Create resource	Required
`PUT`	Update resource	Required
`PATCH`	Partial update	Required
`DELETE`	Remove resource	Optional
`HEAD`	Check existence	None
`OPTIONS`	CORS preflight	None

Request Headers

```
Content-Type: application/json
Accept: application/json
Authorization: Bearer &lt;token&gt;
X-API-Key: &lt;key&gt;
User-Agent: Dynatrace Synthetic
```

```
// List all HTTP monitors
// PREFERRED -- Smartscape. Classic dt.entity.http_check maps to HTTP_MONITOR;
// id_classic still carries the HTTP_CHECK-* id for joining to unmigrated
queries.
smartscapeNodes "HTTP_MONITOR"
| fields name, id, id_classic, http_monitor.type, enabled, frequency
| sort name asc
| limit 50

// Multi-step HTTP requests are separate nodes (HTTP_MONITOR_STEP) joined to
the monitor
// by an edge, not fields of the monitor record. Steps per monitor:
//
// smartscapeNodes "HTTP_MONITOR_STEP"
// | traverse edgeTypes: {"belongs_to"}, targetTypes: {"HTTP_MONITOR"},
direction: forward
// | summarize steps = count(), by:{monitor = name}
// | sort steps desc

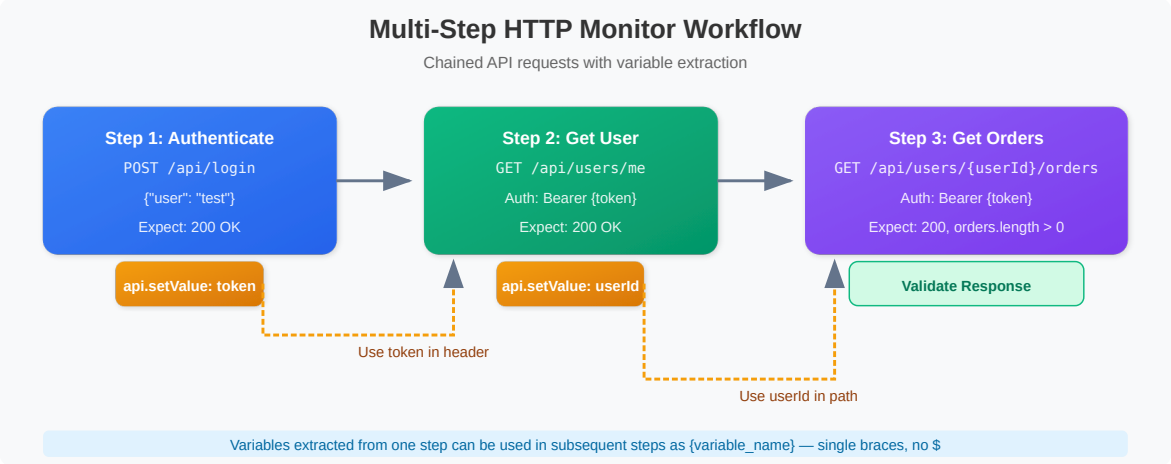
// FALLBACK (classic surface -- deprecated in DQL, supported for as long as
// Dynatrace Classic is supported):
// fetch dt.entity.http_check
// | fields id, entity.name
// | sort entity.name asc
// | limit 50
```

```
// HTTP monitor execution results (last 24h)
// Smartscape fields: dt.entity.synthetic_location / dt.entity.synthetic_test
on synthetic data are
// deprecated ("will be removed in the future") in favor of dt.smartscape.* –
Synthetic events model, 09/21/2026.
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_monitor_execution"
| fields timestamp,
         monitor = monitor.name,
         location = getNodeName(dt.smartscape.synthetic_location),
         state = result.state,
         status_code = result.statistics.response_status_code,
         response_ms = result.statistics.duration / 1ms
| sort timestamp desc
| limit 100
```

3. Multi-Step HTTP Monitors

Chain multiple HTTP requests with data passing between steps:

Multi-Step Workflow Example



Passing Values Between Requests

HTTP monitors have no declarative JSON-path or header extractor. A value is carried forward by a **post-execution script** on the request that returns it, which stores the value with `api.setValue()`:

```
// Post-execution script on request 1 (POST /auth/login)
if (response.getStatusCode() != 200) {
    api.fail("HTTP error: " + response.getStatusCode());
}
var jsonData = JSON.parse(response.getResponseBody());
api.setValue("token", jsonData.access_token);
api.setValue("userId", jsonData.user.id);
```

`response.getResponseBody()` *"returns the first 50 KB of the response body as a string"*, so a value beyond that point cannot be read this way. Response headers are available to the same script through `response.getHeaders()`.

Using Variables

Reference a stored value in any later request field with **single braces and no `$`**:
- URL: `https://api.example.com/users/{userId}`
- Header: `Authorization: Bearer {token}`
- Body: `{"userId": "{userId}"}`

`${userId}` is not substituted — the monitor sends the literal text.

> **Sources:** Pre- and post-execution scripting for HTTP monitors (DT docs) — *"You can also apply the value of a variable previously set using api.setValue() in subsequent HTTP monitor configuration fields using the {variable_name} convention."*; Create and configure an HTTP monitor (DT docs) — *"Pre and post execution scripts allows you to add custom logic between HTTP monitor requests to do things like parsing the response, modifying the request URL, or skipping requests under certain conditions."*

```
// Multi-step HTTP monitor — per-step performance
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_step_execution"
| summarize {
    avg_duration_ms = avg(result.statistics.duration / 1ms),
    success_rate = countIf(result.status.code == 0) * 100.0 / count(),
    executions = count()
  }, by: {monitor.name, step.name}
| fieldsAdd avg_duration_ms = round(avg_duration_ms, decimals: 2)
| fieldsAdd success_rate = round(success_rate, decimals: 2)
| sort avg_duration_ms desc
| limit 30
```

4. Authentication

Supported Authentication Methods

Method	Configuration	Use Case
Basic Auth	Username/password encoded	Legacy APIs
Bearer Token	Header: `Authorization: Bearer <token>`	OAuth2, JWT
API Key	Header: `X-API-Key: <key>`	Third-party APIs
OAuth2	Token endpoint + credentials	Modern APIs
Client Certificate	mTLS	High-security APIs

Credential Vault

Store sensitive credentials securely:

1. **Credential Vault**
2. Add credential (username/password, token, certificate)
3. Reference it in the URL, a header or the body as `{CREDENTIALS_VAULT-<id>|username}` , `{CREDENTIALS_VAULT-<id>|password}` or `{CREDENTIALS_VAULT-<id>|token}` — the documented form is *"{|token} , {|username} , or {|password}"*

OAuth2 Flow Example

```
Request 1: Get token
    POST https://auth.example.com/oauth/token
    Body: grant_type=client_credentials
          &amp;client_id={CREDENTIALS_VAULT-&lt;id&gt;|username}
          &amp;client_secret={CREDENTIALS_VAULT-&lt;id&gt;|password}
    Post-execution script:
          var jsonData = JSON.parse(response.getResponseBody());
          api.setValue("bearerToken", jsonData.access_token);

Request 2: API call
    GET https://api.example.com/data
    Header: Authorization: Bearer {bearerToken}
```

> **Sources:** Create and configure an HTTP monitor (DT docs), Pre- and post-execution scripting for HTTP monitors (DT docs) — its OAuth example parses the token response with `JSON.parse` and stores the token with `api.setValue`.

5. Response Validation

HTTP Status Validation

Status Range	Meaning	Default Behavior
2xx	Success	Pass
3xx	Redirect	Follow/Pass
4xx	Client Error	Fail
5xx	Server Error	Fail

Content Validation

Type	Description	Example
Contains	Text present	`"status": "ok"`
Not Contains	Text absent	`"error"`
Regex	Pattern match	`"id":\s*\d+`

Text and regex rules are evaluated over the first 50 KB of the response body.

Checking JSON Responses

JSON-path assertions are **not** an HTTP-monitor rule type. The documented rule types are *"Response status code validation Text validation, where the first 50 KB of the response body is checked for a string of text or a regular expression SSL certificate expiry validation"*, and when several are configured, *"HTTP status code validation is the most important, followed by text and regular expression validation, and finally by SSL certificate expiry validation."*

For a structured check, either match the JSON as text or regex (for example `"status":\s*"success"`), or assert in a post-execution script, where `api.fail()` *"marks the request as failed, providing the message as the reason, and marks the monitor execution as failed"*:

```
var d = JSON.parse(response.getResponseBody());
if (d.status !== "success") {
    api.fail("status=" + d.status);
}
if (!d.data || !d.data.users || d.data.users.length === 0) {
    api.fail("users array is empty");
}
```

> **Sources:** Create and configure an HTTP monitor (DT docs), Pre- and post-execution scripting for HTTP monitors (DT docs).

```
// HTTP status code distribution
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_monitor_execution"
| summarize count = count(), by: {monitor.name,
result.statistics.response_status_code}
| sort count desc
| limit 30
```

```
// Failed HTTP requests with error details
// Smartscape fields: dt.entity.synthetic_location / dt.entity.synthetic_test
on synthetic data are
// deprecated ("will be removed in the future") in favor of dt.smartscape.* —
Synthetic events model, 09/21/2026.
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_monitor_execution"
| filter result.state == "FAIL"
| fields timestamp,
         monitor = monitor.name,
         location = getNodeName(dt.smartscape.synthetic_location),
         status = result.status.message,
         status_code = result.status.code,
         detail = result.status.details
| sort timestamp desc
| limit 50
```

6. SSL Certificate Monitoring

HTTP monitors automatically check SSL certificates:

Certificate Checks

"Accept any SSL certificate —by default, HTTP monitors fail when SSL certificates are invalid." The docs list what counts as invalid: self-signed, expired, inactive, lifetime greater than 398 days, missing hostname, invalid or incomplete chain, untrusted certificate authority, insecure signature algorithm.

Expiration Problems

"Generate a problem if the SSL certificate expires within the next n days" — n is at most 100 — raises a problem *"without failing the monitor or affecting monitor availability"*. Pick n to leave time to renew; in community practice 30 days is a common setting, with the § 6 query as an early view.

> **Sources:** Create and configure an HTTP monitor (DT docs) — *"Accept any SSL certificate —by default, HTTP monitors fail when SSL certificates are invalid."*

```
// SSL certificate expiration status (per monitor)
// peer_certificate_expiry_date is carried on http_step_execution records for
HTTPS targets
fetch dt.synthetic.events, from: now() - 6h
| filter event.type == "http_step_execution"
| filter isNotNull(result.statistics.peer_certificate_expiry_date)
| summarize { certificate_expiry =
takeMax(result.statistics.peer_certificate_expiry_date) }, by: {monitor.name}
| fieldsAdd days_until_expiry = round((certificate_expiry - now()) / 1h / 24,
decimals: 1)
| sort days_until_expiry asc
| limit 20
```

7. Analyzing HTTP Results

```
// HTTP monitor availability summary (by monitor)
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_monitor_execution"
| summarize {
    total = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
  }, by: {monitor.name}
| fieldsAdd availability_pct = round((successful * 100.0) / total, decimals:
2)
| sort availability_pct asc
| limit 30
```

```
// Response time percentiles by monitor (successful executions)
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_monitor_execution"
| filter result.state == "SUCCESS"
| summarize {
    p50 = percentile(result.statistics.duration, 50),
    p95 = percentile(result.statistics.duration, 95),
    p99 = percentile(result.statistics.duration, 99),
    max = max(result.statistics.duration),
    executions = count()
  }, by: {monitor.name}
| fieldsAdd p50_ms = round(p50 / 1ms, decimals: 1),
            p95_ms = round(p95 / 1ms, decimals: 1),
            p99_ms = round(p99 / 1ms, decimals: 1),
            max_ms = round(max / 1ms, decimals: 1)
| fieldsRemove p50, p95, p99, max
| sort p95_ms desc
| limit 20
```

```
// HTTP timing breakdown (DNS, TCP connect, TLS, TTFB) — from step records
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_step_execution"
| filter result.status.code == 0
| summarize {
    avg_dns_ms     = round(avg(result.statistics.host_name_resolution_time) /
1ms, decimals: 2),
    avg_connect_ms = round(avg(result.statistics.tcp_connect_time) / 1ms,
decimals: 2),
    avg_tls_ms     = round(avg(result.statistics.tls_handshake_time) / 1ms,
decimals: 2),
    avg_ttfb_ms    = round(avg(result.statistics.time_to_first_byte) / 1ms,
decimals: 2),
    avg_total_ms   = round(avg(result.statistics.duration) / 1ms, decimals:
2),
    executions = count()
  }, by: {monitor.name, step.name}
| sort avg_total_ms desc
| limit 20
```

```
// Response time trend over time (last 7 days)
fetch dt.synthetic.events, from: now() - 7d
| filter event.type == "http_monitor_execution"
| filter result.state == "SUCCESS"
| makeTimeseries {
    avg_response_ms = avg(result.statistics.duration / 1ms),
    p95_response_ms = percentile(result.statistics.duration / 1ms, 95)
  }, interval: 1h
```

Summary

In this notebook, you learned:

✅ **HTTP monitor types** - Single request vs multi-step
✅ **Request configuration** - Methods, headers, body
✅ **Multi-step workflows** - Values passed with `api.setValue()` and referenced as `{variable}`
✅ **Authentication** - Basic, Bearer, OAuth2, API keys
✅ **Response validation** - Status codes, text/regex, post-execution script checks
✅ **SSL monitoring** - Certificate expiration via

`result.statistics.peer_certificate_expiry_date`
✅ **Analysis queries** - Availability, percentiles, timing breakdown

---

Next Steps

Continue to **SYNTH-04: Private Locations** to learn about running monitors from your own infrastructure.

References

- Create and configure an HTTP monitor (DT docs)
- HTTP monitor metrics in Synthetic on Grail (DT docs)
- Credential vault (DT docs)
- Pre- and post-execution scripting for HTTP monitors (DT docs)
- Synthetic app (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.